

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника» _____

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКЕ
(2025/2026 учебный год)

Андреев Михаил Анатольевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника» _____

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

Андреев Михаил Анатольевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный
интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	2	29.06.26 - 29.06.26	
2	Подбор и изучение материала по теме работы	15	30.06.26 – 02.07.26	
3	Разработка алгоритма	43	02.07.26 – 06.07.26	
4	Описание алгоритма и программы	18	06.07.26 – 08.07.26	
5	Тестирование	5	08.07.26 – 08.07.26	
6	Получение и анализ результатов	10	08.07.26 – 10.07.26	
7	Оформление отчёта	15	10.07.26– 12.07.26	
	Общий объём часов	108		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Андреев Михаил Анатольевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Андреев М.А. в составе бригады выполнял практическое задание «Шейкерная сортировка». На первоначальном этапе им были изучены и проанализированы особенности алгоритма шейкерной сортировки, выбран язык программирования С для программной реализации вычислительного ядра. В процессе выполнения практики Андреевым М.А. была разработана функция cocktailSort, реализовано добавление подсчета количества перестановок элементов, проведено тестирование алгоритма и проверка корректности сортировки на маленьких массивах. Также им были подготовлены текстовое объяснение принципа работы алгоритма, детальная блок-схема его логики и оформлен соответствующий раздел отчета по практике.

Бакалавр Андреев М.А. "___" _____ 2026 г.

Руководитель Карамышева Н.С. "___" _____ 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Андреев Михаил Анатольевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Андреев М.А. решал следующие задачи: программная реализация ядра алгоритма шейкерной сортировки, интеграция механизма подсчета перестановок, верификация корректности алгоритма на малых массивах, а также графическое моделирование и текстовое описание логики его работы.

За период прохождения практики были освоены основные понятия и технологии программирования, изучены основные инструменты языка C. И получены следующие результаты: разработана вычислительная функция cocktailSort, успешно реализован модуль подсчета количества перестановок, построена детальная блок-схема алгоритма и подготовлено его подробное объяснение для отчета. Во время выполнения работы Андреев М.А. показал себя ответственным, добросовестным студентом, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике; программированию.

За выполнение работы Андреев М.А. заслуживает оценки «_____».
Руководитель практики к.т.н., Карамышева Н.С. «_____» _____ 2026 г.

Содержание

Введение	2
1 Постановка задачи	3
1.1 Достоинства алгоритма шейкерной сортировки	3
1.2 Недостатки алгоритма шейкерной сортировки	3
1.3 Типичные сценарии применения данного алгоритма	3
2 Выбор решения	4
3 Описание программы	5
4. Схемы программы	7
4.1 Блок-схема программы	7
4.2 Блок-схема алгоритма	8
5 Тестирование программы	9
6 Отладка	10
7 Совместная разработка	11
Заключение	15
Список используемой литературы	16
Приложение А. Листинг программы	17

Введение

Сортировка данных является одной из наиболее распространённых операций при обработке информации. Она применяется практически во всех областях программирования и вычислительной техники, поскольку позволяет упорядочить данные для их дальнейшего хранения, анализа и быстрого поиска.

На сегодняшний день существует большое количество алгоритмов сортировки, отличающихся сложностью реализации, временем выполнения и областью применения. Одним из таких алгоритмов является шейкерная сортировка (Cocktail Sort). Она представляет собой усовершенствованный вариант пузырьковой сортировки и отличается тем, что проход по массиву выполняется в двух направлениях: сначала слева направо, затем справа налево. Благодаря этому элементы быстрее перемещаются к своим окончательным позициям, что позволяет уменьшить количество проходов массива по сравнению с классической пузырьковой сортировкой.

Целью данной практической работы, выполняемой в составе бригады, является разработка консольной программы на языке программирования C, реализующей алгоритм шейкерной сортировки массива. В рамках командного проекта я отвечал за программный код и корректность алгоритма: реализовал функцию `cocktailSort`, добавил подсчёт количества перестановок элементов, выполнил проверку сортировки на маленьких массивах, а также подготовил блок-схему и текстовое объяснение логики метода.

В процессе выполнения практики мной были получены навыки разработки алгоритмического ядра на языке C, проведения модульного тестирования программ, построения графических схем логики, использования системы контроля версий GitHub для совместной работы в команде и оформления отчетной технической документации.

1 Постановка задачи

В рамках учебной практики нашей бригаде необходимо разработать программу, реализующую алгоритм шейкерной сортировки массива.

Программа должна обеспечивать возможность загрузки массива целых чисел из CSV-файла либо генерации случайного массива заданного размера. После получения исходных данных необходимо выполнить сортировку массива методом шейкерной сортировки, определить время выполнения алгоритма и подсчитать количество перестановок элементов.

После завершения сортировки программа должна вывести результаты на экран и сохранить отсортированный массив в CSV-файл, имя которого задаётся пользователем.

При разработке программы бригадой использован язык программирования C и реализован удобный интерфейс в виде консольного меню. Моя роль в команде заключалась в написании кода сортировки и обеспечении корректности алгоритма: мной создана функция `cocktailSort`, добавлен подсчет перестановок, выполнена проверка на маленьких массивах, а также подготовлены блок-схема и объяснение алгоритма в отчете. Совместная разработка велась с использованием репозитория GitHub.

1.1 Достоинства шейкерной сортировки

- ~ простой алгоритм реализации;
- ~ является стабильной сортировкой, сохраняя порядок одинаковых элементов;
- ~ выполняет проход массива одновременно в двух направлениях.

1.2 Недостатки алгоритма шейкерной сортировки

- квадратичная временная сложность $O(n^2)$;
- большое количество сравнений и перестановок элементов;
- низкая эффективность при работе с массивами большого размера;
- уступает современным алгоритмам сортировки (Quick Sort, Merge Sort, Heap Sort) по производительности.

1.3 Типичные сценарии применения данного алгоритма

- ~ сортировка списков студентов по фамилии или среднему баллу;
- ~ упорядочивание товаров по цене, весу или количеству;

- ~ обработка небольших таблиц данных;
- ~ учебные проекты по изучению алгоритмов сортировки.
- ~ сортировка результатов измерений;

2 Выбор решения

Для реализации поставленной перед нашей бригадой задачи был выбран язык программирования С и среда разработки Microsoft Visual Studio.

Язык программирования С является одним из наиболее распространённых языков программирования общего назначения. Он отличается высокой производительностью, простотой работы с памятью и предоставляет программисту широкие возможности при реализации алгоритмов обработки данных. Благодаря этим особенностям язык С широко используется при разработке системного и прикладного программного обеспечения, а также при изучении основ программирования.

В качестве среды разработки использовалась Microsoft Visual Studio. Данная среда предоставляет удобные средства редактирования исходного кода, компиляции программы, поиска ошибок и отладки приложений. Использование Visual Studio значительно упростило процесс написания и тестирования разработанного мной кода.

Для хранения исходного кода и организации совместной разработки использовался сервис GitHub. Применение системы контроля версий позволило распределить задачи между участниками бригады, зафиксировать в удалённом репозитории разработанную мной функцию `cocktailSort` с добавленным подсчётом перестановок, а также координировать общую сборку проекта.

В ходе выполнения практической работы программа была реализована в виде консольного приложения. Такой способ реализации позволил полностью сосредоточить внимание на корректности работы алгоритма шейкерной сортировки, его проверке на маленьких массивах, построении блок-схемы и подготовке текстового объяснения для отчёта, не требуя создания графического пользовательского интерфейса.

3 Описание программы

При запуске программы выводится меню из трёх пунктов:

- Загрузить массив из CSV-файла.
- Сгенерировать массив случайных чисел.
- Выйти из программы.

```
printf("\n===== МЕНЮ =====\n");
printf("1. Загрузить массив из CSV файла\n");
printf("2. Сгенерировать массив случайных чисел\n");
printf("3. Выйти из программы\n");
printf("Выберите действие (1-3): ");
```

Пользователю требуется выбрать тот пункт, который ему нужен. При выборе пункта 1 программа запрашивает имя исходного CSV-файла, из которого будут считаны данные. При выборе пункта 2 пользователю необходимо ввести количество элементов массива, а также минимальное и максимальное значения диапазона для генерации случайных чисел.

```
printf("Введите количество элементов массива (макс. %d): ", MAX_SIZE);
scanf("%d", &n);
printf("Введите минимальное значение диапазона: ");
scanf("%d", &min_val);
printf("Введите максимальное значение диапазона: ");
scanf("%d", &max_val);
```

После того как данные были введены, генерируется массив из случайных чисел в заданном диапазоне. Если был выбран пункт 1, массив загружается из указанного CSV-файла (числа в файле должны быть разделены запятыми). Исходный массив выводится на экран.

```
void generateRandomArray(int arr[], int n, int min_val, int max_val)
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = min_val + rand() % (max_val - min_val + 1);
    }
}
```

Далее над этими данными выполняется шейкерная сортировка (Cocktail Sort). Алгоритм представляет собой усовершенствованную версию пузырьковой сортировки: массив просматривается попеременно слева направо и справа налево. При проходе слева направо наибольший элемент перемещается в конец неотсортированной части, а при проходе справа налево наименьший — в начало. Границы неотсортированной части (left и right) сужаются после каждого прохода. Процесс повторяется до тех пор, пока не произойдёт ни одной перестановки за полный цикл, что означает завершение

сортировки.

После этого отсортированный массив выводится на экран и сохраняется в CSV-файл, имя которого задаёт пользователь.

```
int saveCSV(const char* filename, int arr[], int n)
{
    FILE* file = fopen(filename, "w");
    if (file == NULL)
        return 0;

    for (int i = 0; i < n; i++)
    {
        fprintf(file, "%d", arr[i]);
        if (i < n - 1)
            fprintf(file, ",");
    }

    fclose(file);
    return 1;
}
```

Программа также осуществляет подсчёт количества перестановок элементов массива (переменная `swar_count` в функции `cocktailSort`) и измеряет время, которое заняла сортировка, с помощью функций `clock()` из библиотеки `time.h`. Результаты замеров выводятся на экран.

```
clock_t start_time = clock();
long long total_swaps = cocktailSort(arr, n);
clock_t end_time = clock();
double time_taken = (double)(end_time - start_time) / CLOCKS_PER_SEC;
При выборе пункта меню 3 программа завершает выполнение.
```

Подробный алгоритм работы программы и функции шейкерной сортировки представлен в подразделе 4.1 на рисунках 1, 2.

Листинг программы приведён в приложении А.

4. Схемы программы

4.1 Блок-схема программы

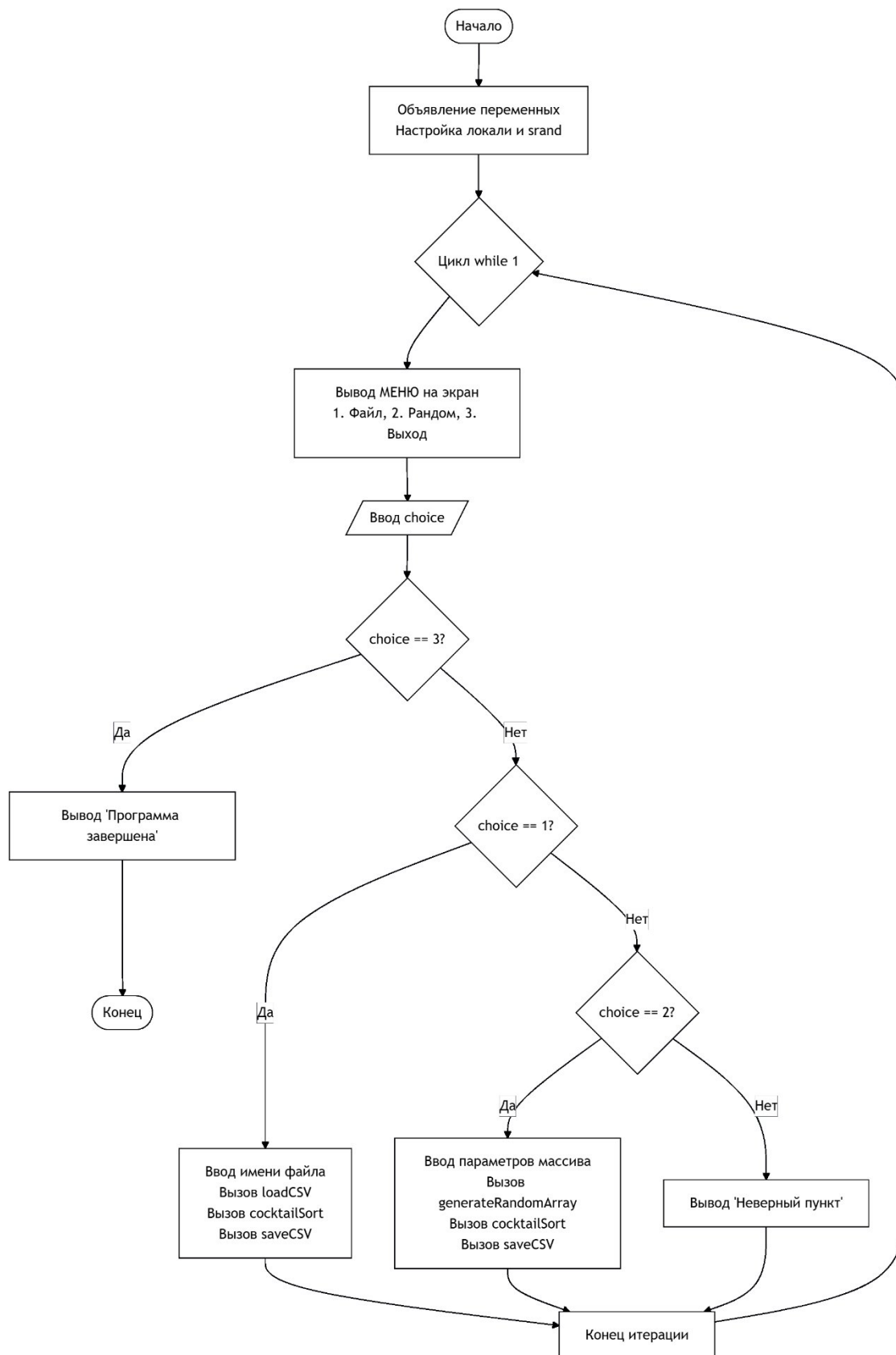


Рисунок 1 - Блок-схема программы

4.2 Блок-схема алгоритма

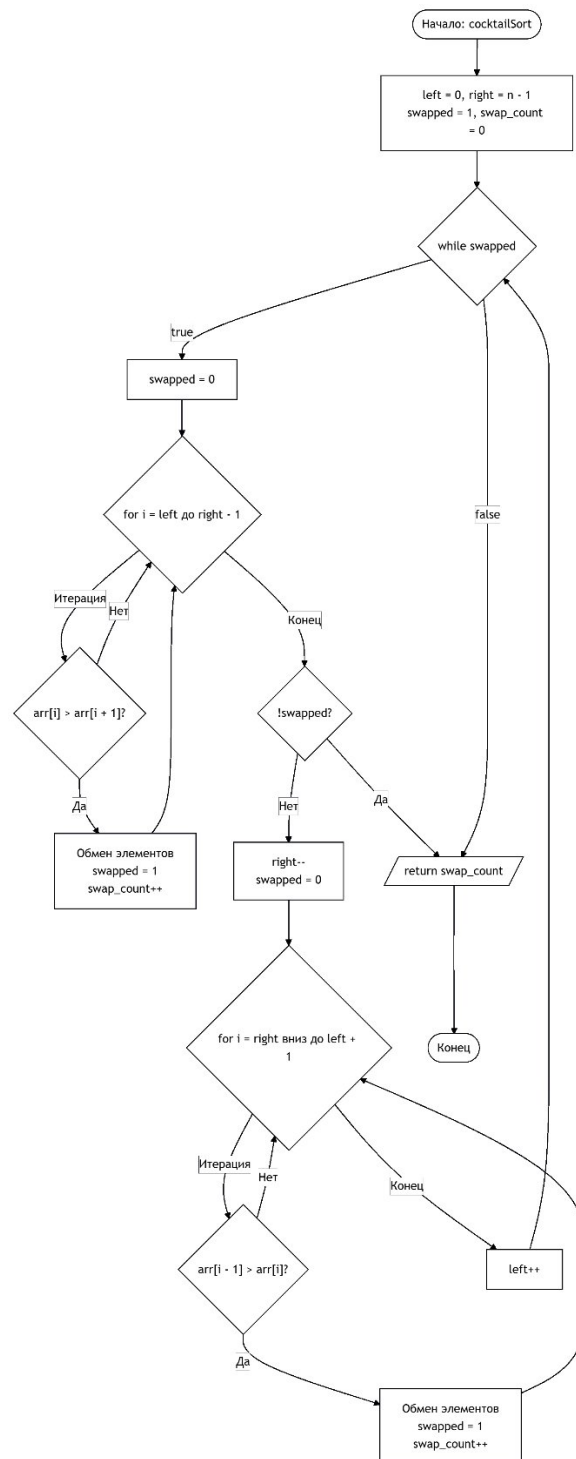


Рисунок 2 - Блок-схема алгоритма с подключенными файлами

5 Тестирование программы

Тестирование показало, что с увеличением количества элементов пропорционально увеличивается время работы программы, ниже представлен график результатов тестирования:



Рисунок 3 – Результаты тестирования

6 Отладка

В качестве среды разработки была выбрана Microsoft Visual Studio, которая предоставляет все необходимые средства для написания, компиляции и отладки программ на языке С.

Для проверки корректности работы программы использовались встроенные средства отладки Visual Studio: точки останова, пошаговое выполнение и просмотр значений локальных переменных. Это позволило контролировать выполнение программы на каждом этапе и своевременно обнаруживать возможные ошибки.

В процессе отладки проверялась работа основных функций программы:

- загрузка массива из CSV-файла (функция loadCSV);
- генерация случайного массива (функция generateRandomArray);
- выполнение шейкерной сортировки (функция cocktailSort);
- измерение времени работы алгоритма (функции clock из time.h);
- сохранение отсортированного массива в CSV-файл (функция saveCSV).

Особое внимание уделялось обработке ошибочных ситуаций:

- попытка открытия несуществующего файла — loadCSV возвращает -1, программа выводит сообщение об ошибке;
- пустой файл — возвращается -2;
- некорректные данные в CSV-файле — возвращается -3;
- ввод пользователем недопустимого размера массива (меньше 1 или больше MAX_SIZE) — программа выводит предупреждение и возвращает в меню;
- неверный ввод в меню (не число) — программа завершается с соответствующим сообщением.

Во всех перечисленных случаях программа корректно обнаруживала ошибки и выводила информативные сообщения, не допуская аварийного завершения.

Для анализа выполнения программы использовалась команда «Шаг с заходом» (*Step Into*), позволяющая последовательно выполнять инструкции и контролировать работу отдельных функций. В результате проведённой отладки все выявленные недочёты были устранены, после чего программа успешно протестирована и подготовлена к эксплуатации.

7 Совместная разработка

Для удобства совместной разработки был использован сервис github. Определили задачи проекта, назначили приоритет задачам. Разделили роли, назначили исполнителей задачи.

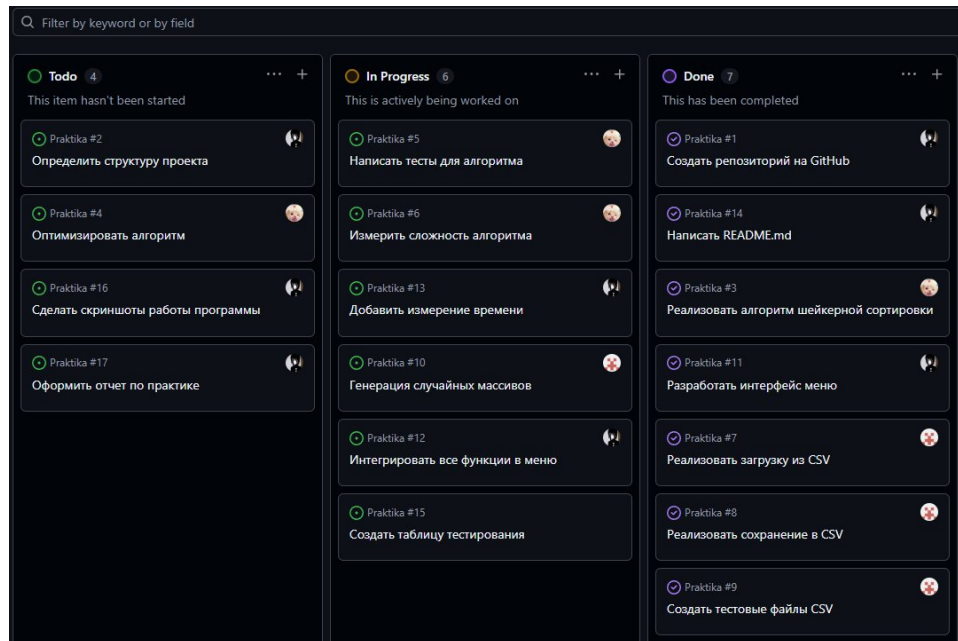


Рисунок 3 – Определение задач проекта и распределение ролей

Корректировали статус задач по мере выполнения.

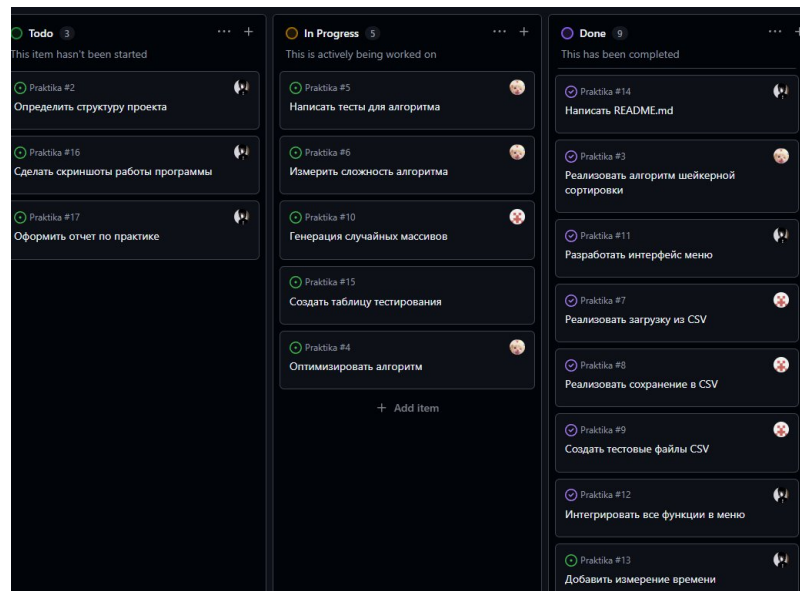


Рисунок 4 – Корректирование задач

Были подключены тестовые файлы и файл README, а также было написано меню, которое помогает ориентироваться в программе, это было зафиксировано и загружено на удаленный репозиторий Github, на ветку main.

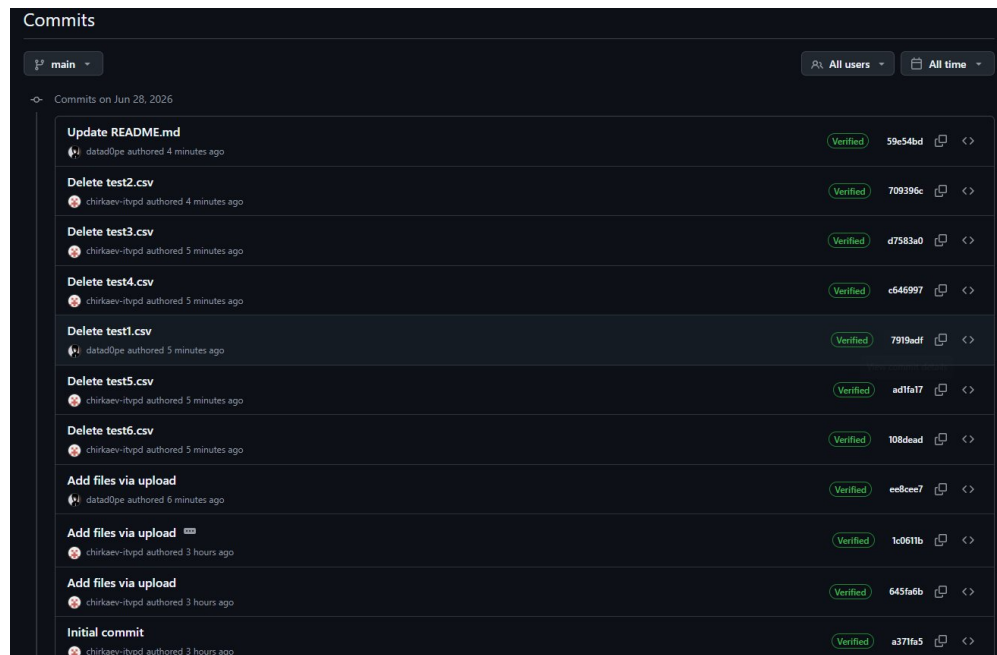


Рисунок 5 – Созданные коммиты

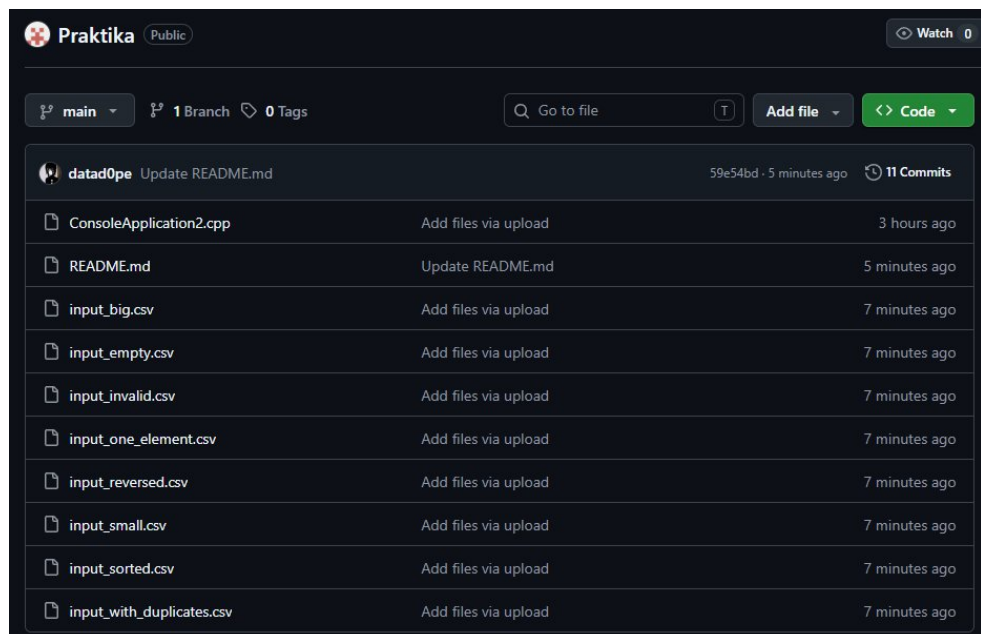


Рисунок 6– Ветка main

Ссылка на удаленный репозиторий: <https://github.com/chirkaev-itvpd/Praktika.git>:

Заключение

В ходе выполнения учебной практики была разработана консольная программа на языке C, реализующая алгоритм шейкерной сортировки (Cocktail Sort) массива целых чисел.

Программа обеспечивает загрузку данных из CSV-файла, генерацию случайного массива в заданном диапазоне, выполнение сортировки, измерение времени работы алгоритма, подсчёт количества перестановок элементов и сохранение отсортированного массива в выходной CSV-файл.

В рамках закреплённых за мной обязанностей были выполнены следующие задачи:

- реализована функция шейкерной сортировки `cocktailSort` с подсчётом количества перестановок;
- разработано пользовательское меню программы;
- реализовано измерение времени выполнения алгоритма с использованием функций `clock()` из библиотеки `time.h`;
- проведено тестирование программы на различных наборах данных;
- подготовлены таблицы результатов тестирования;
- оформлен файл `README`, организована структура репозитория GitHub и подготовлен отчёт по практике.

Проведённое тестирование подтвердило корректную работу всех функций программы. Полученные результаты показали, что время выполнения алгоритма закономерно увеличивается с ростом количества элементов массива, что соответствует теоретической оценке сложности шейкерной сортировки.

В процессе выполнения практической работы были закреплены знания языка программирования C, освоены навыки работы с CSV-файлами, проведено тестирование программного обеспечения, получен опыт использования системы контроля версий Git и оформления технической документации.

Список используемой литературы

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2019. — 304 с.
2. Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2018. — 832 с.
3. Вирт Н. Алгоритмы и структуры данных. — СПб.: Питер, 2019. — 304 с.

Приложение А. Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <time.h>
#include <windows.h>

#define MAX_SIZE 10000

long long cocktailSort(int arr[], int n)
{
    int left = 0;
    int right = n - 1;
    int swapped = 1;
    long long swap_count = 0;

    while (swapped)
    {
        swapped = 0;

        for (int i = left; i < right; i++)
        {
            if (arr[i] > arr[i + 1])
            {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                swapped = 1;
                swap_count++;
            }
        }

        if (!swapped)
            break;

        right--;
        swapped = 0;

        for (int i = right; i > left; i--)
        {
            if (arr[i - 1] > arr[i])
            {
                int temp = arr[i];
                arr[i] = arr[i - 1];
                arr[i - 1] = temp;
                swapped = 1;
                swap_count++;
            }
        }
    }
}
```

```

        left++;
    }
    return swap_count;
}

int loadCSV(const char* filename, int arr[])
{
    FILE* file = fopen(filename, "r");
    if (file == NULL)
    {
        return -1;
    }

    char line[65536];

    if (fgets(line, sizeof(line), file) == NULL)
    {
        fclose(file);
        return -2;
    }

    fclose(file);

    int count = 0;
    char* token = strtok(line, ",");

    while (token != NULL)
    {
        if (count >= MAX_SIZE) break;

        char* endptr;
        long value = strtol(token, &endptr, 10);

        while (*endptr == ' ' || *endptr == '\n' || *endptr == '\r' ||
*endptr == '\t')
        {
            endptr++;
        }

        if (*endptr != '\0')
        {
            return -3;
        }

        arr[count++] = (int)value;
        token = strtok(NULL, ",");
    }

    return count;
}

int saveCSV(const char* filename, int arr[], int n)

```

```

{
    FILE* file = fopen(filename, "w");
    if (file == NULL)
        return 0;

    for (int i = 0; i < n; i++)
    {
        fprintf(file, "%d", arr[i]);
        if (i < n - 1)
            fprintf(file, ",");
    }

    fclose(file);
    return 1;
}

void printArray(int arr[], int n)
{
    if (n > 20) {
        for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
        printf("... ");
        for (int i = n - 10; i < n; i++) printf("%d ", arr[i]);
        printf("\n(Всего элементов: %d. Вывод ограничен)\n", n);
    }
    else {
        for (int i = 0; i < n; i++)
        {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}

void generateRandomArray(int arr[], int n, int min_val, int max_val)
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = min_val + rand() % (max_val - min_val + 1);
    }
}

int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    srand((unsigned int)time(NULL));

    int arr[MAX_SIZE];
    int n = 0;
    int choice;

```

```

char input_filename[256] = "input.csv";
char output_filename[256] = "output.csv";

while (1)
{
    printf("\n===== МЕНЮ =====\n");
    printf("1. Загрузить массив из CSV файла\n");
    printf("2. Сгенерировать массив случайных чисел\n");
    printf("3. Выйти из программы\n");
    printf("Выберите действие (1-3): ");

    if (scanf("%d", &choice) != 1) {
        printf("Ошибка ввода чисел! Выход.\n");
        break;
    }

    if (choice == 3) {
        printf("Программа завершена.\n");
        break;
    }

    if (choice == 1) {
        printf("Введите имя исходного файла (например, input.csv):
");
        scanf("%255s", input_filename);

        n = loadCSV(input_filename, arr);

        if (n == -1) {
            printf("Ошибка: не удалось открыть файл %s\n",
input_filename);
            continue;
        }
        if (n == -2) {
            printf("Ошибка: файл %s пуст.\n", input_filename);
            continue;
        }
        if (n == -3) {
            printf("Ошибка: некорректные данные в файле %s.\n",
input_filename);
            continue;
        }
        printf("Массив успешно загружен из файла!\n");
    }
    else if (choice == 2) {
        int min_val, max_val;
        printf("Введите количество элементов массива (макс. %d): ",
MAX_SIZE);
        scanf("%d", &n);
        if (n <= 0 || n > MAX_SIZE) {
            printf("Некорректный размер массива. Возврат в
меню.\n");
            continue;
        }
    }
}

```



```

    }

    printf("Введите минимальное значение диапазона: ");
    scanf("%d", &min_val);
    printf("Введите максимальное значение диапазона: ");
    scanf("%d", &max_val);

    if (min_val > max_val) {
        int t = min_val; min_val = max_val; max_val = t;
    }

    generateRandomArray(arr, n, min_val, max_val);
    printf("Массив случайных чисел успешно сгенерирован!\n");
}
else {
    printf("Неверный пункт меню. Попробуйте снова.\n");
    continue;
}

printf("\nИсходный массив:\n");
printArray(arr, n);

printf("\nСортировка алгоритмом Cocktail Sort...\n");

clock_t start_time = clock();
long long total_swaps = cocktailSort(arr, n);
clock_t end_time = clock();

double time_taken = (double)(end_time - start_time) /
CLOCKS_PER_SEC;

printf("Отсортированный массив:\n");
printArray(arr, n);

printf("\n-----\n");
printf("Время работы алгоритма: %.6f секунд.\n", time_taken);
printf("Количество перестановок: %lld\n", total_swaps);
printf("-----\n");

printf("Введите название результирующего файла для сохранения\n(например, output.csv): ");
scanf("%255s", output_filename);

if (saveCSV(output_filename, arr, n)) {
    printf("Результат успешно сохранен в файл %s\n",
output_filename);
}
else {
    printf("Ошибка сохранения в файл %s\n", output_filename);
}
}

return 0;

```

}

